School of Information and Computer Technology
Sirindhorn International Institute of Technology, Thammasat University
CSS326 Database Programming Laboratory

Laboratory #9: Transactions & Concurrency Control

Objectives:

- To explain why some operations must happen all-or-nothing.
- To use START TRANSACTION, COMMIT and ROLLBACK in SQL.
- To run a transaction in Node.js with a single pooled connection in mysql2.
- To roll back on both errors and broken business rules.
- To make a multi-step route, such as enrolling with limited seats, safe and atomic.

1. Recap and Goal

Every single SQL statement already succeeds or fails as a whole: one INSERT either happens or it doesn't. But real operations often take several statements that belong together. Enrolling a student, for example, means recording the enrolment **and** taking one seat from the course. Those two changes must both happen, or neither. This lab introduces transactions, which group statements so they all commit together or all roll back. We continue with the students, courses and enrolments schema, and add a small `seats` column to courses for the examples.

2. The Problem: Partial Failure

Suppose you insert the enrolment, then try to reduce the course's free seats, and the second step fails because of a bad value, a constraint, or a crash. Now there is an enrolment for a seat that was never taken, and the data contradicts itself. Worse, the server might stop between the two statements and leave the change half-done. We need a way to say: do all of these together, or undo everything and leave the database as if nothing had started.

3. What Is a Transaction?

A transaction groups several statements into one unit of work. If every statement succeeds, you **COMMIT** and all the changes become permanent at once. If anything goes wrong, you **ROLLBACK** and every change in the group is undone. From the outside the whole thing is atomic: other users never see a half-finished state.

4. ACID in One Section

- **Atomicity:** all of the statements happen, or none do.
- **Consistency:** the database moves from one valid state to another, respecting its rules.
- **Isolation:** concurrent transactions don't trip over each other's half-done work.
- **Durability:** once committed, a change survives even a crash.

5. Setup: the lab9 Database and a Seats Column

Run `sql/lab9_setup.sql` from the starter project once in MySQL Workbench. It creates a fresh `lab9` database with the students, courses and enrolments from Lab 3. It also adds a UNIQUE key (a student can't enrol in the same course twice) and foreign keys (an enrolment must point to a real student and a real course).

Then give each course a number of **free** seats with ALTER TABLE, the statement you met in Lab 3. Each course holds 30 students. `DEFAULT 30` is right for new courses, but some students are already enrolled. A second statement subtracts them, so `seats` really means free seats. A course is full when its seats reach zero. Enrolling takes one seat, and un-enrolling gives it back.

```
ALTER TABLE courses ADD COLUMN seats INT NOT NULL DEFAULT 30;

-- seats = FREE seats: subtract the students already enrolled
UPDATE courses c
   SET seats = 30 - (SELECT COUNT(*) FROM enrolments e WHERE e.course_id = c.id);

SELECT id, title, seats FROM courses;  -- course 1 has 4 students -> 26
```

> **Note**
> Run the ALTER TABLE on its own, **not** inside a transaction. MySQL commits automatically before and after a statement that changes a table's structure, so ROLLBACK can't undo it.

6. Transactions in SQL

You can try a transaction directly in MySQL Workbench before writing any code. Begin with START TRANSACTION and run your statements. Then either COMMIT to keep them or ROLLBACK to discard them all. Run the example below with ROLLBACK and confirm nothing changed; then run it again with COMMIT.

```
START TRANSACTION;

UPDATE courses SET seats = seats - 1 WHERE id = 1;
INSERT INTO enrolments (student_id, course_id) VALUES (3, 1);

COMMIT;  -- make both changes permanent
-- ROLLBACK;  -- ...or undo both, as if nothing happened
```

7. Transactions in mysql2: One Connection

A transaction lives on a single connection, so every statement in it must run on that same connection. That means you can't use the pool directly: `pool.execute()` may hand each statement to a different connection. Instead, borrow one connection with `pool.getConnection()`, begin the transaction on it, run your statements, and then commit or roll back. Finally, release the connection back to the pool. Always do this, even if something failed.

8. The getConnection / beginTransaction / commit / rollback Pattern

Memorise this shape: **borrow, begin, try/commit, catch/rollback, finally/release.** Every transaction you write follows this skeleton.

```
const conn = await pool.getConnection();
try {
  await conn.beginTransaction();
  await conn.execute(/* statement 1 */);
  await conn.execute(/* statement 2 */);
  await conn.commit();
} catch (err) {
  await conn.rollback();
  throw err;
} finally {
  conn.release();
}
```

9. A Worked Example: Enrolling with a Seat

Take a seat and record the enrolment together. The trick is to make the seat update its own guard. `UPDATE … WHERE seats > 0` only changes a row when a seat is actually free, so its `affectedRows` tells you whether one was available. In a controller, map each outcome to a status code: **201** on success, **409 Conflict** when the course is full, and **500** on an unexpected error. The data is consistent in every case.

```
exports.enrol = async (req, res) => {
  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    const [u] = await conn.execute(
      'UPDATE courses SET seats = seats - 1 WHERE id = ? AND seats > 0',
      [req.body.course_id]);
    if (u.affectedRows === 0) {
      await conn.rollback();
      return res.status(409).json({ error: 'course is full' });
    }
    await conn.execute(
      'INSERT INTO enrolments (student_id, course_id) VALUES (?, ?)',
      [req.params.id, req.body.course_id]);
    await conn.commit();
    res.status(201).json({ ok: true });
  } catch (err) {
    await conn.rollback();
    res.status(500).json({ error: err.message });
  } finally {
    conn.release();
  }
};
```

10. Rolling Back on Errors and on Broken Business Rules

A rollback isn't only for crashes; you also roll back when a rule is broken. "The course is full" is a rule. There is no database error, but the operation must not proceed. Above, we detect it (`affectedRows === 0`), roll back, and return immediately with a meaningful 409 instead of a vague 500. Another way



is to `throw` inside the try block. That sends you to the catch, which rolls back the whole group, but then the catch must turn that error into a 409. Either way, the same mechanism handles both unexpected failures and deliberate refusals.

11. Isolation and the Last Seat

Imagine two students trying to grab the one remaining seat at the very same moment. Without care, both could read seats = 1 and both enrol, overselling the course. Running each enrolment as a transaction, with `UPDATE … WHERE seats > 0` as the guard, prevents this. The database serialises the two updates: one wins the seat, and the other's guard matches no row, so it rolls back. Letting a guarded UPDATE do the checking is safer than reading the count and deciding in JavaScript.

12. When to Use a Transaction

Use a transaction whenever a single logical action spans more than one statement and a half-finished result would be wrong. Money is the classic case: debit one account and credit another, never one without the other. Inventory is another: place an order and reduce stock, or cancel an order and restore it. If a half-finished result would be wrong or unsafe, wrap the steps in a transaction.

13. Good Habits

- Always borrow a single connection for a transaction, and release it in `finally`.
- Roll back on broken rules as well as on errors; never leave a transaction open.
- Keep transactions short: do the work and commit. Don't hold a connection while waiting on other things.
- Let a guarded UPDATE (`WHERE seats > 0`) do the checking, so two requests can't both win.
- Map each outcome to a clear status code in your controller.

14. Lab Exercises (complete during class)

Getting started

Download and unzip **Lab9_Exercise_Starter.zip**. It contains:

```
Lab9_Exercise_Starter/
  db.js                    connection pool (edit user / password)
  server.js                Express app + front-end on http://localhost:3000
  public/index.html        front-end for Exercises 2 and 3
  routes/                  courses.js, students.js
  controllers/             course-, student- and enrolmentController.js
  sql/lab9_setup.sql       creates the lab9 database (Section 5)
  sql/exercise1.sql        Exercise 1 script with TODOs
  scripts/race.js          fires many enrol requests at once
  requests.http            ready-made test requests
```

- In Workbench, run `sql/lab9_setup.sql` once.
- Open `db.js` and change `user` and `password` to your own MySQL login. The database name is already `lab9`.

Then open a terminal **in the project folder** (in VS Code: Terminal → New Terminal) and run the two npm commands below.

```
npm install      # once: downloads express and mysql2 into node_modules/
npm run dev      # starts the server
                 #   > node --watch server.js
                 #   Lab 9 server on http://localhost:3000
                 # Ctrl + C stops the server
```

`npm install` reads the dependencies listed in `package.json` and downloads them into a `node_modules` folder. You only need it once per project, or again after `package.json` changes. `npm run dev` runs the script named **dev** in `package.json`, which is `node --watch server.js`. It starts the Express server and **restarts it automatically every time you save a .js file**, so you don't have to stop and start it after each change. (`npm start` runs plain `node server.js` without the automatic restart.) Leave the terminal open while you work; press **Ctrl + C** to stop the server.

- Open **http://localhost:3000** in your browser. The front-end shows every course's free seats, has an Enrol / Cancel panel for one student, a **Last-seat race** button, and a log of every response with its status code.
- You can also test with `requests.http` (VS Code REST Client) or Postman.

If the server doesn't start, read the first error line in the terminal:

Error message	What it means / what to do
`Cannot find module 'express'`	You skipped `npm install`, or ran it in a different folder.
`Access denied for user 'root'`	Wrong login: set your own `user` / `password` in `db.js`.
`Unknown database 'lab9'`	Run `sql/lab9_setup.sql` in Workbench first.
`Unknown column 'seats'`	Do Exercise 1 (ALTER TABLE + UPDATE) before using the API.
`EADDRINUSE … :3000`	The server is already running in another terminal; stop it with Ctrl + C.

Exercise 1: Run a transaction by hand. Open `sql/exercise1.sql` in Workbench. Add the seats column with ALTER TABLE and sync it with the existing enrolments (Section 5). Then run START TRANSACTION, reduce course 1's seats, and enrol student 3 in course 1. ROLLBACK and confirm both changes vanished. Repeat with COMMIT, and note that the data only changes for good after COMMIT.

```
START TRANSACTION;
UPDATE courses SET seats = seats - 1 WHERE id = 1;
INSERT INTO enrolments (student_id, course_id) VALUES (3, 1);
ROLLBACK;  -- then try again with COMMIT;
```

Exercise 2: A transactional enrol. In `controllers/enrolmentController.js`, the `enrol` route (`POST /students/:id/enrolments`, body `{"course_id": 3}`) works, but it isn't a transaction. Rewrite it using `pool.getConnection()` and the begin / commit / rollback / release pattern. Guard the seat with `WHERE seats > 0` and return 409 when the course is full. To see the seats run out, first give one course only two seats in Workbench: `UPDATE courses SET seats = 2 WHERE id = 3;`. Then press **Send enrol requests for ALL students at once** in the front-end (or run `npm run race`

`-- 3 1 14`), which sends 14 enrol requests at the same moment. Run it before and after your change; afterwards, exactly two enrolments succeed and seats never go below zero.

```
const conn = await pool.getConnection();
try {
  await conn.beginTransaction();
  /* take a seat (guarded UPDATE), then insert the enrolment */
  await conn.commit();
  res.status(201).json({ ok: true });
} catch (err) {
  await conn.rollback();
  res.status(500).json({ error: err.message });
} finally {
  conn.release();
}
```

Exercise 3: Cancel atomically. Implement `unenrol` (`DELETE /students/:id/enrolments/:courseId`). It removes the enrolment and gives the seat back, both inside one transaction, so a seat is never lost or double-counted. If no enrolment was deleted, roll back and return 404. In the front-end, enrol a student, press **Cancel** next to that course, and confirm the course's seats return to where they started.

```
await conn.beginTransaction();
const [d] = await conn.execute(
  'DELETE FROM enrolments WHERE student_id = ? AND course_id = ?',
  [studentId, courseId]);
// if d.affectedRows === 0 -> rollback and return 404
await conn.execute(
  'UPDATE courses SET seats = seats + 1 WHERE id = ?', [courseId]);
await conn.commit();
```

Exercise 4: Prove the rollback (if time). Make the second statement fail and show that the first change was undone. For example, enrol student 999, who doesn't exist, so the foreign key rejects the INSERT. Or enrol student 1 in course 1 again, which the UNIQUE key rejects. Watch the seats in the front-end (or `GET /courses`) before and after: the count is unchanged after the failed attempt.

15. Where We're Going Next

Your data now stays consistent, even when an action takes several steps. As your tables grow, the next question is speed: why is a particular query slow, and how do we make it fast? In **Lab 10** we look at indexing and at reading a query plan with EXPLAIN, so your queries stay quick as the data grows.